UNIT 1: INTRODUCTION TO SDLC MODELS

What is a software?

Software is a set of instructions that tells a computer, web-based application, or other devices what to do. It helps the device understand what to do and how to do it. Through software, you can interact with the device in ways they couldn't before. For example, with software, you can use a computer to create graphics, edit videos, create music, and play games without having to know how the hardware works. The software makes interacting with computers easier and allows us access to new features and capabilities not possible without it.

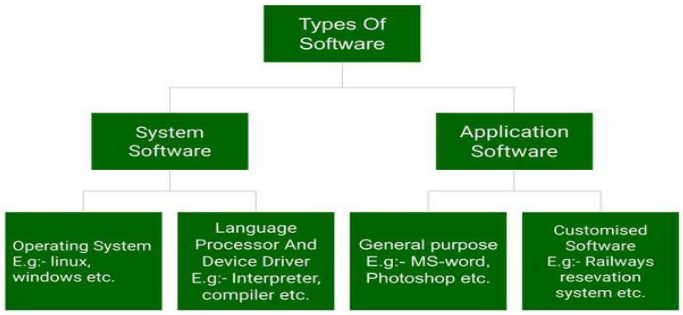
Software and its Types

In a computer system, the software is basically a set of instructions or commands that tell a computer to do a specific task that serves its users.

- A software is not a physical thing like hardware, it rather makes the hardware work as per user requirements by giving instructions.
- Examples of software are MS-Word, MS-Excel, PowerPoint and Web Browser

Types of Software

The chart below describes he types of software:



Above is the diagram of types of software. Now we will briefly describe each type and its subtypes:

1. **System Software**
    - Operating System (like Windows and Linux)
    - Language Processor
    - Device Driver
2. **Application Software**
    - General Purpose Software
    - Customize Software

- Utility Software

System Software

System software is software that directly operates the computer hardware and provides the basic functionality to the users as well as to the other software to operate smoothly.

- System software basically controls a computer's internal functioning and also controls hardware devices such as monitors, printers, and storage devices, etc.
- It is like an interface between hardware and user applications, it helps them to communicate with each other because hardware understands machine language(i.e. 1 or 0) whereas user applications are work in human-readable languages like English, Hindi, German, etc.

Types of System Software

It has two subtypes which are:

1. **Operating System:** It is the main program of a computer system. When the computer system ON it is the first software that loads into the computer's memory. Basically, it manages all the resources such as computer memory, CPU, printer, hard disk, etc., and provides an interface to the user, which helps the user to interact with the computer system.
2. **Language Processor:** As we know that system software converts the human-readable language into a machine language and vice versa. So, the conversion is done by the language processor. It converts programs written in high-level programming languages like Java, C, C++, Python, etc(known as source code), into sets of instructions that are easily readable by machines(known as object code or machine code).
3. **Device Driver:** A device driver is a program or software that controls a device and helps that device to perform its functions. Every device like a printer, mouse, modem, etc. needs a driver to connect with the computer system eternally. So, when you connect a new device with your computer system, first you need to install the driver of that device so that your operating system knows how to control or manage that device.

Features of System Software

Let us discuss some of the features of System Software:

- Closer to the computer system.
- Written in a low-level language in general.
- Difficult to design and understand.
- Fast in speed(working speed).
- Less interactive for the users in comparison to application software.

Application Software

Software that performs special functions or provides functions that are much more than the basic operation of the computer is known as application software.

- Application software is designed to perform a specific task for end-users. It is a product or a program that is designed only to fulfill end-users' requirements.
- Examples include word processors, spreadsheets, database management, inventory, payroll programs, etc.

Types of Application Software

There are different types of application software and those are:

1. **General Purpose Software:** This type of application software is used for a variety of tasks and it is not limited to performing a specific task only. For example, MS-Word, MS-Excel, PowerPoint, etc.
2. **Customized Software:** This type of application software is used or designed to perform specific tasks or functions or designed for specific organizations. For example, railway reservation system, airline reservation system, invoice management system, etc.
3. **Utility Software:** This type of application software is used to support the computer infrastructure. It is designed to analyze, configure, optimize and maintains the system, and take care of its requirements as well. For example, antivirus, disk fragmenter, memory tester, disk repair, disk cleaners, registry cleaners, disk space analyzer, etc.

Features of Application Software

Let us discuss some of the features of Application Software:

- Perform more specialized tasks like word processing, spreadsheets, email, etc.
- Mostly, the size of the software is big, so it requires more storage space.
- More interactive for the users, so it is easy to use and design.
- Easy to design and understand.
- Written in a high-level language in general.

Difference Between System Software and Application Software

Now, let us discuss some difference between system software and application software:

System Software	Application Software
It is designed to manage the resources of the computer system, like memory and process management, etc.	It is designed to fulfill the requirements of the user for performing specific tasks.

System Software	Application Software
Written in a low-level language.	Written in a high-level language.
Less interactive for the users.	More interactive for the users.
System software plays vital role for the effective functioning of a system.	Application software is not so important for the functioning of the system, as it is task specific.
It is independent of the application software to run.	It needs system software to run.

What Is a Software Product?

Software Products is any Software or any system which has been developed, tested and maintained for the specific purpose to solve that problem.

In certain cases, software products may be part of system products where hardware, as well as software, is delivered to a customer. Software products are produced with the help of the software process. The software process is a way in which we produce software.

Based on why the software product are developed, the Software product mainly divided into the two parts which are follows

Types of Software Products

Software products fall into two broad categories:

1. Generic products: Generic products are stand-alone systems that are developed by a production unit and sold on the open market to any customer who can buy them.

Products like **Microsoft Office** or **Google Chrome**. They are ready to use right out of the box and are designed to meet the general needs of many users. Whether you're working on a document or browsing the web, these tools are flexible enough to suit a wide range of tasks.

2. Customized Products: Customized products are the systems that are commissioned by a particular customer. Some contractor develops the software for that customer.

These are designed specifically for a particular business or set of users. Imagine a software system built just for managing employee data in a company or a custom CRM system that's perfectly aligned with the way a business operates.

Characteristics of Software Product

Software Product should possess the following important characteristics:

1. **Efficiency:** The software should not make wasteful use of system resources such as memory and processor cycles.
2. **Maintainability:** It should be possible to evolve the software to meet the changing requirements of customers.
3. **Dependability:** It is the flexibility of the software that ought to not cause any physical or economic injury in the event of system failure. It includes a range of characteristics such as reliability, security, and safety.
4. **In time:** Software should be developed well in time.
5. **Within Budget:** The software development costs should not be overrun, and they should be within the budgetary limit.
6. **Functionality:** The software system should exhibit the proper functionality, i.e., it should perform all the functions it is supposed to perform.
7. **Adaptability:** The software system should have the ability to adapted to a reasonable extent with the changing requirements.

What is a Good Software?

Software engineering is the process of designing, developing, and maintaining software systems. Good software meets the needs of its users, performs its intended functions reliably, and is easy to maintain.

1. There are several characteristics of good software that are commonly recognized by software engineers, which are important to consider when developing a software system.
2. These characteristics include functionality, usability, reliability, performance, security, maintainability, reusability, scalability, and testability.

Characterisitics of Good Software

1. **Functionality:** The software meets the requirements and specifications that it was designed for, and it behaves as expected when it is used in its intended environment.
2. **Usability:** The software is easy to use and understand, and it provides a positive user experience.
3. **Reliability:** The software is free of defects and it performs consistently and accurately under different conditions and scenarios.
4. **Performance:** The software runs efficiently and quickly, and it can handle large amounts of data or traffic.
5. **Security:** The software is protected against unauthorized access and it keeps the data and functions safe from malicious attacks.
6. **Maintainability:** The software is easy to change and update, and it is well-documented, so that it can be understood and modified by other developers.

7. **Reusability:** The software can be reused in other projects or applications, and it is designed in a way that promotes code reuse.
8. **Scalability:** The software can handle an increasing workload and it can be easily extended to meet the changing requirements.
9. **Testability:** The software is designed in a way that makes it easy to test and validate, and it has a comprehensive test coverage.

Software Engineering

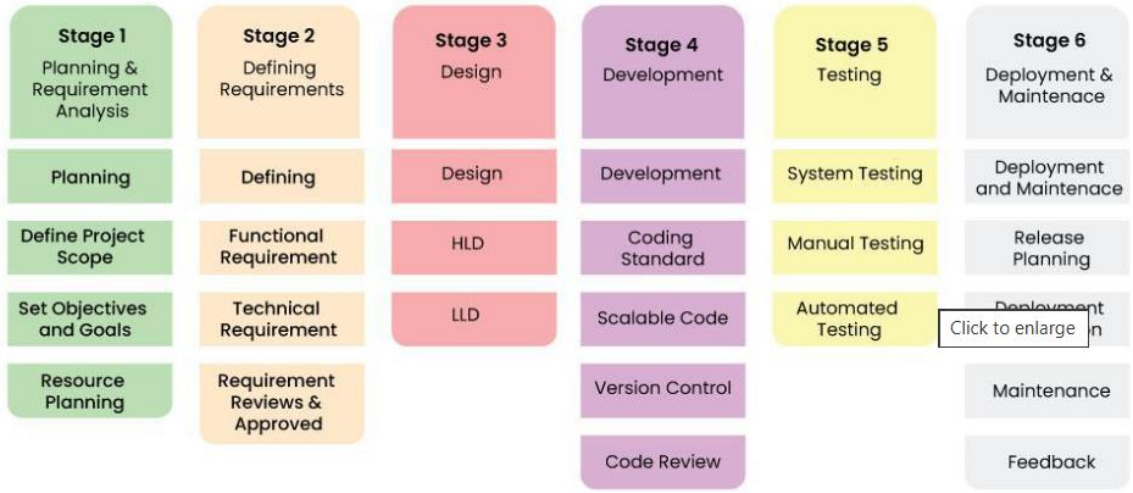
- Software Engineering is the systematic process of designing, developing, testing, and maintaining software using techniques like requirements analysis, design, testing, and maintenance.
- It ensures software is high-quality, reliable, and maintainable, especially for large software systems.
- Software Engineering improves efficiency, quality, and budget management, helping deliver software on time, within budget, and meeting all requirements
- It continuously adopting new tools and methodologies to handle complexity and evolving technologies.

Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.

The goal of the SDLC is to produce high-quality software that meets or exceeds customer expectations, reaches completion within time and cost estimates, and is efficient to maintain.

Stages of the Software Development Life Cycle



Stage 1: Planning and Requirement Analysis

This is the most fundamental stage. Before writing code, the team must define what they are building and why.

- **Activities:** Feasibility studies (technical, operational, economic), resource allocation, project scheduling, and cost estimation.
- **Output:** Project Plan, Feasibility Report.
- **Key Players:** Senior Engineers, Project Managers, Stakeholders.

Stage 2: Defining Requirements (SRS)

Once the plan is approved, the specific product requirements must be defined and documented. This is done through the **Software Requirement Specification (SRS)** document.

- **Activities:** Gathering detailed functional and non-functional requirements from customers.
- **Output:** SRS Document (The "Bible" for the development team).
- **Key Players:** Business Analysts (BAs), Product Owners

Stage 3: Designing Architecture

The requirements are turned into a technical blueprint. This phase defines the overall system architecture and technology stack.

- **High-Level Design (HLD):** Defines the architecture, database design, and relationships between modules.
- **Low-Level Design (LLD):** Defines the logic of individual components, API interfaces, and database tables.
- **Output:** Design Document Specification (DDS).

- **Key Players:** System Architects, Lead Developers.

Stage 4: Development (Coding)

This is the longest phase where the actual building takes place. Developers write code based on the Design Document.

- **Activities:** Coding, Code Reviews, Unit Testing, and Static Code Analysis.
- **Tools:** Compilers, Debuggers, IDEs (VS Code, IntelliJ), Version Control (Git).
- **Output:** Source Code, Executable Software.
- **Key Players:** Developers (Frontend, Backend, Full Stack).

Stage 5: Testing

Once the code is written, it moves to the Quality Assurance (QA) team. The goal is to find bugs before the customer does.

Types of Testing:

- **Unit Testing:** Testing individual functions.
- **Integration Testing:** Ensuring modules work together.
- **System Testing:** Testing the entire application flow.
- **User Acceptance Testing (UAT):** Verifying the software meets business needs.

Output: Bug Reports, Test Cases, Quality Report.

Key Players: QA Engineers, Testers.

Stage 6: Deployment

The software is released to the end-users. In modern DevOps environments, this is often automated via CI/CD pipelines.

- **Activities:** Setting up production environments, deploying code, and smoke testing the live environment.
- **Output:** Live Application.
- **Key Players:** DevOps Engineers, Release Managers.

Stage 7: Maintenance

The cycle doesn't end at deployment. Software must be maintained to remain useful.

- **Activities:** Bug fixing, upgrading libraries, performance tuning, and adding new features.
- **Output:** Patches, Updates, New Versions.
- **Key Players:** Support Engineers, Developers.